20 Agentic Design Patterns (Part 4)

A comprehensive guide to 20 agentic design patterns that separate pros from beginners, based on a Google engineer's 400-page book. Practical patterns you can use today with plain English explanations.

Back

Navigation

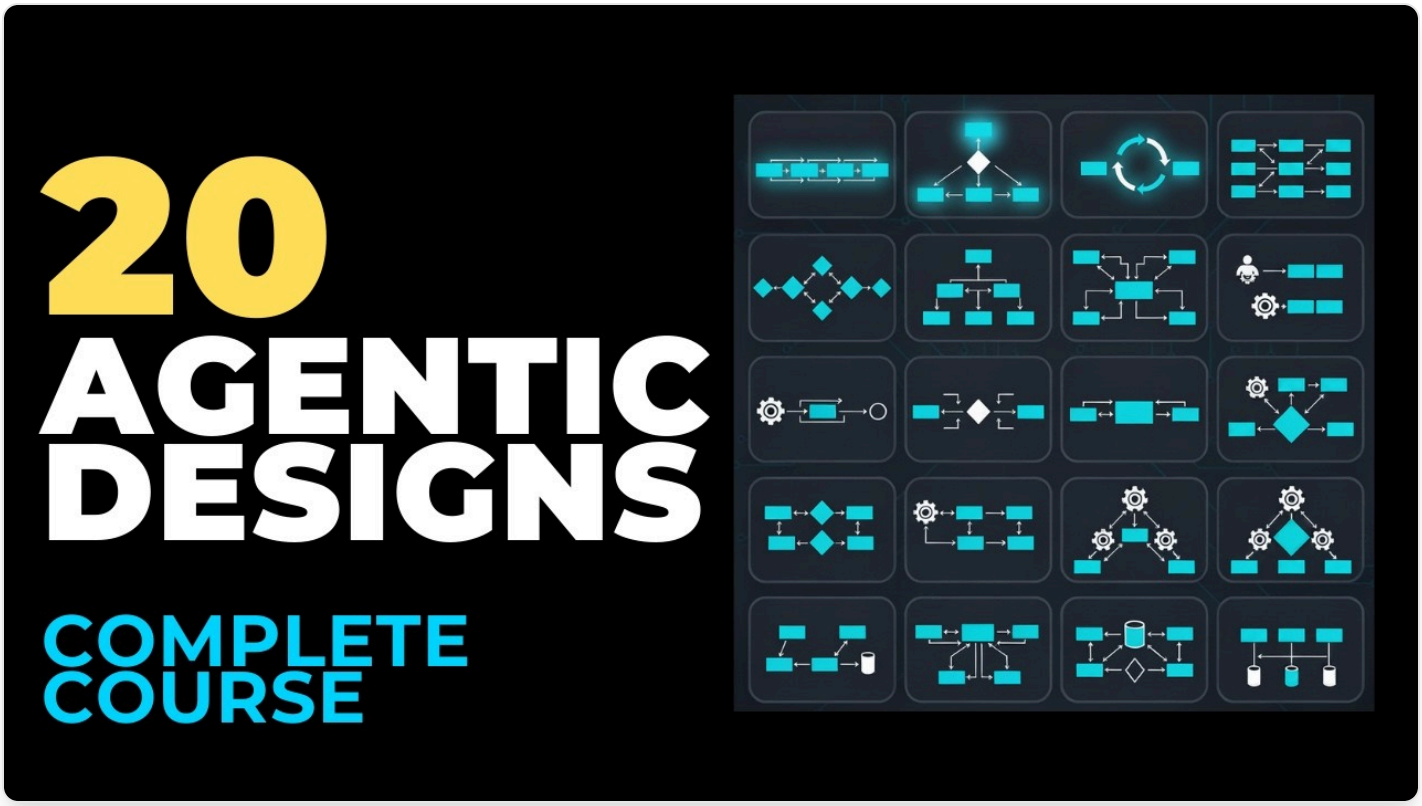
This guide is split into 4 parts for better performance:

- Part 1: Chapters 1-5 - Prompt Chaining, Routing, Parallelization, Reflection, Tool Use
- Part 2: Chapters 6-10 - Planning, Multi-Agent Collaboration, Memory Management, Learning and Adaptation, Goal Setting and Monitoring
- Part 3: Chapters 11-15 - Exception Handling and Recovery, Human in the Loop, Knowledge Retrieval (RAG), Inter-Agent Communication, Resource-Aware Optimization
- Part 4: Chapters 16-20 - Reasoning Techniques, Evaluation and Monitoring, Guardrails and Safety Patterns, Prioritization, Exploration and Discovery

Originally video talking about agentic systems.

You can help out the author who broke down the 400-page manual published by the Google engineer here.

Link not affiliated.



Chapter 16: Reasoning Techniques

TLDDR: Choosing the right method for the right problem. So chain of thought for step-by-step logic. Tree of thought, a very interesting technique. It's actually one of my favorite for different use cases that need creativity and imagination for exploring multiple paths. So this one is like solving a puzzle by trying different strategies until one finally works. While you might not find this

these methods work is very fundamental to actually making this work. So this is on the end of the spectrum in my opinion. This is advanced.

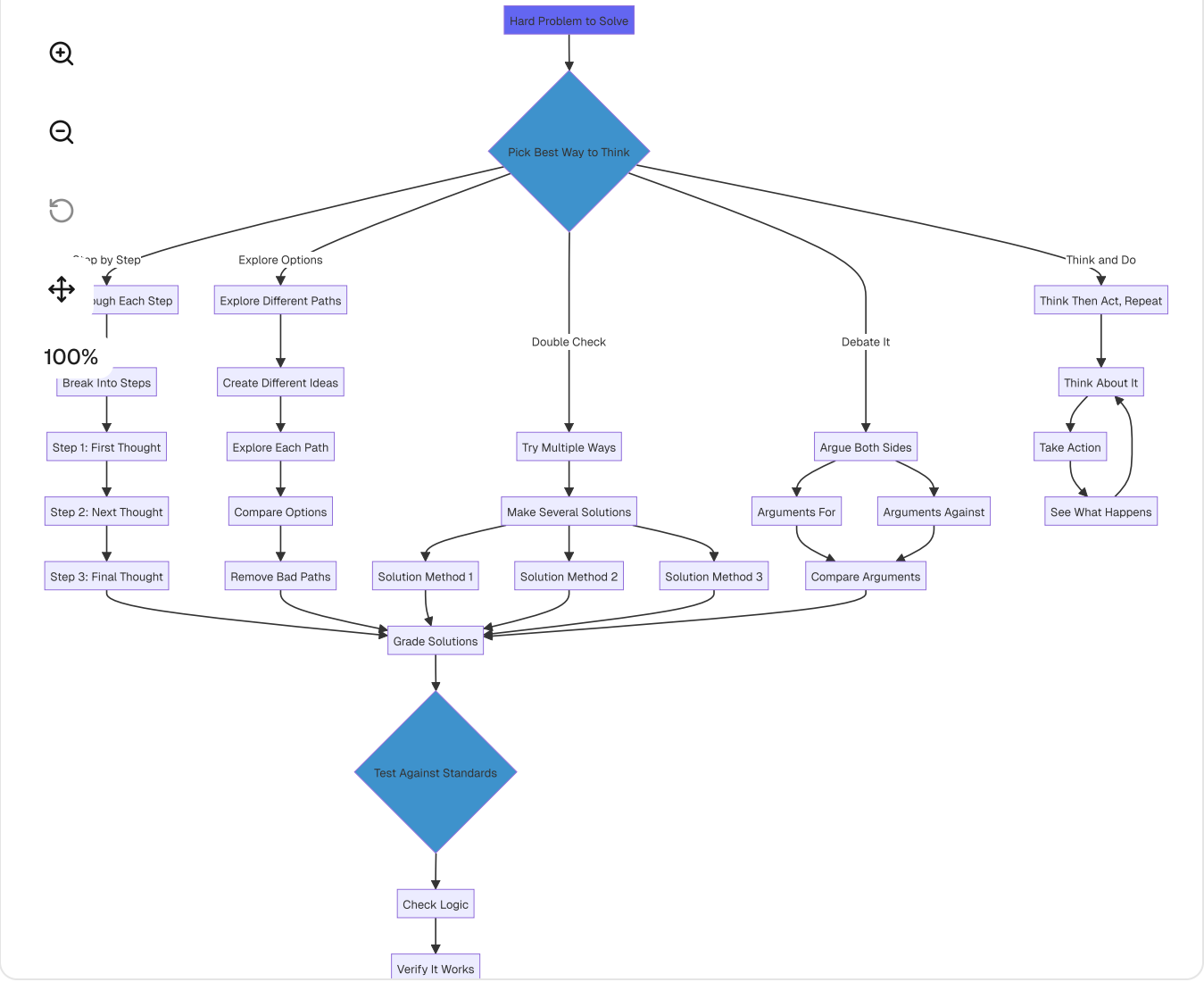
When to Use Where It Fits

When to Use

- **Complex problem-solving**: Multi-step logical challenges
- **Mathematical reasoning**: Problems requiring systematic thinking
- **Strategic planning**: Evaluating multiple approaches
- **Critical analysis**: Deep examination of options
- **Decision making**: Weighing alternatives systematically
- **Creative exploration**: Generating diverse solutions

How It Works

MermaidZoom MMD



How do reasoning techniques work?

Pick best way to think - chain of thought (step by step), tree of thought (explore paths), self-consistency (try multiple ways), debate (argue both sides), ReAct (think then act)

What's tree of thought?

Generate branches of thought, explore each path, evaluate which is most viable, then prune (cut off dead branches). Great for creativity and

What's debate method?

Have proponent agent and opponent agent. Like mini parliament - two agents go back and forth until one wins, exchange arguments, decide best path forward

How do you choose?

Score all solutions based on rubric, run tests, validate logic, rank candidates. Select best one or combine methods (like prompt chaining + tree of thought)

Is this practical?

Advanced technique. Only for very complex things - mathematical reasoning, strategic planning at scale. Nine times out of ten you won't need it. Highly experimental unless you have bandwidth or free time

Pros Cons

Pros

- **Improved accuracy**: Systematic thinking reduces errors
- **Transparency**: Clear reasoning traces
- **Exploration**: Considers multiple solution paths
- **Robustness**: Multiple methods provide validation
- **Learning**: Reasoning traces help improvement
- **Flexibility**: Different techniques for different problems
- **Quality**: Higher quality solutions through deliberation

Real-World Examples

Math Problem Solver | Business Advisor | Code Architect | Medical Diagnosis | Legal Analyzer | Investme

Mathematical Problem Solver

- Chain-of-Thought for step-by-step solutions
- Self-consistency checking multiple approaches
- Tree-of-Thoughts exploring solution branches
- Validation through different methods
- Clear explanation generation

Chapter 17: Evaluation and Monitoring

TLDDR: Setting up quality gates and golden tests before deployment and continuously monitoring accuracy, performance, cost and drift in production. What drift is is when you have the same model or the same suite of models output one response but over time that response degrades or gets worse or more unpredictable. Think of it as a factory quality control system that checks products at every stage.

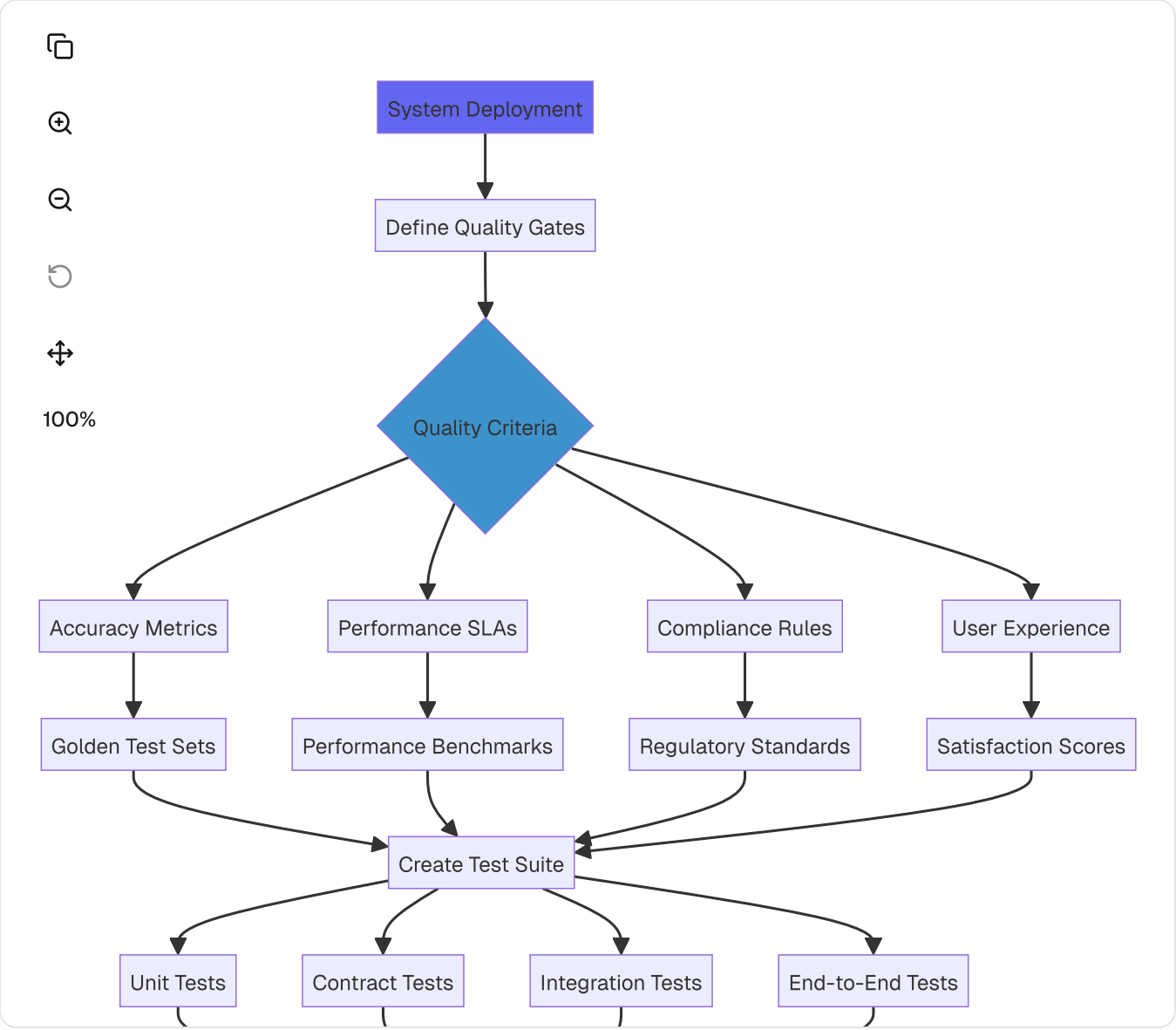
When to Use | Where It Fits

When to Use

- **Production systems**: Any system requiring reliability
- **Quality assurance**: Ensuring consistent performance
- **Compliance requirements**: Meeting regulatory standards
- **Performance optimization**: Identifying bottlenecks
- **Cost management**: Tracking resource usage

How It Works

MermaidZoom MMD



How does evaluation and monitoring work?

Set up quality gates before deployment - accuracy metrics, performance SLAs, compliance rules, user experience

What are quality gates?

Golden test sets, performance benchmarks, regulatory standards, satisfaction scores. Create test suite - unit tests, contract tests, integration tests, end-to-end tests

Then what?

Instrument system - distributed traces, system metrics, cost tracking, latency monitoring. Collect all the data

How do you analyze it?

Detect drift (when model output degrades over time), find regressions (things that deviate from the mean), spot anomalies, identify trends. Check if thresholds are breached

What if thresholds are breached?

Alert teams, investigate issue. Decide: rollback changes, deploy fix, or accept and document decision. Verify recovery

How do you keep improving?

Periodic audits, review performance, update eval sets, refresh tests. Continuous improvement loop

Pros Cons

- **Reliability**: Early detection of issues
- **Performance visibility**: Clear system insights
- **Quality assurance**: Consistent output standards
- **Cost control**: Resource usage tracking
- **Compliance**: Audit trail maintenance
- **Improvement data**: Metrics guide optimization
- **User trust**: Transparent performance metrics

Real-World Examples

E-commerce | Customer Service | Trading System | Content Moderation | Medical Diagnosis | Code Gene

E-commerce Recommendation Engine

- Click-through rate monitoring
- Conversion tracking
- A/B test evaluation
- Latency monitoring
- Cost per recommendation
- Drift detection in user preferences

Chapter 18: Guardrails and Safety Patterns

TLDDR: Checking all the inputs for harmful content, personal info or injection attacks. This is much more top of funnel of that entire infrastructure. You're classifying risk levels and apply appropriate controls. The main analogy here is airport security where you have multiple

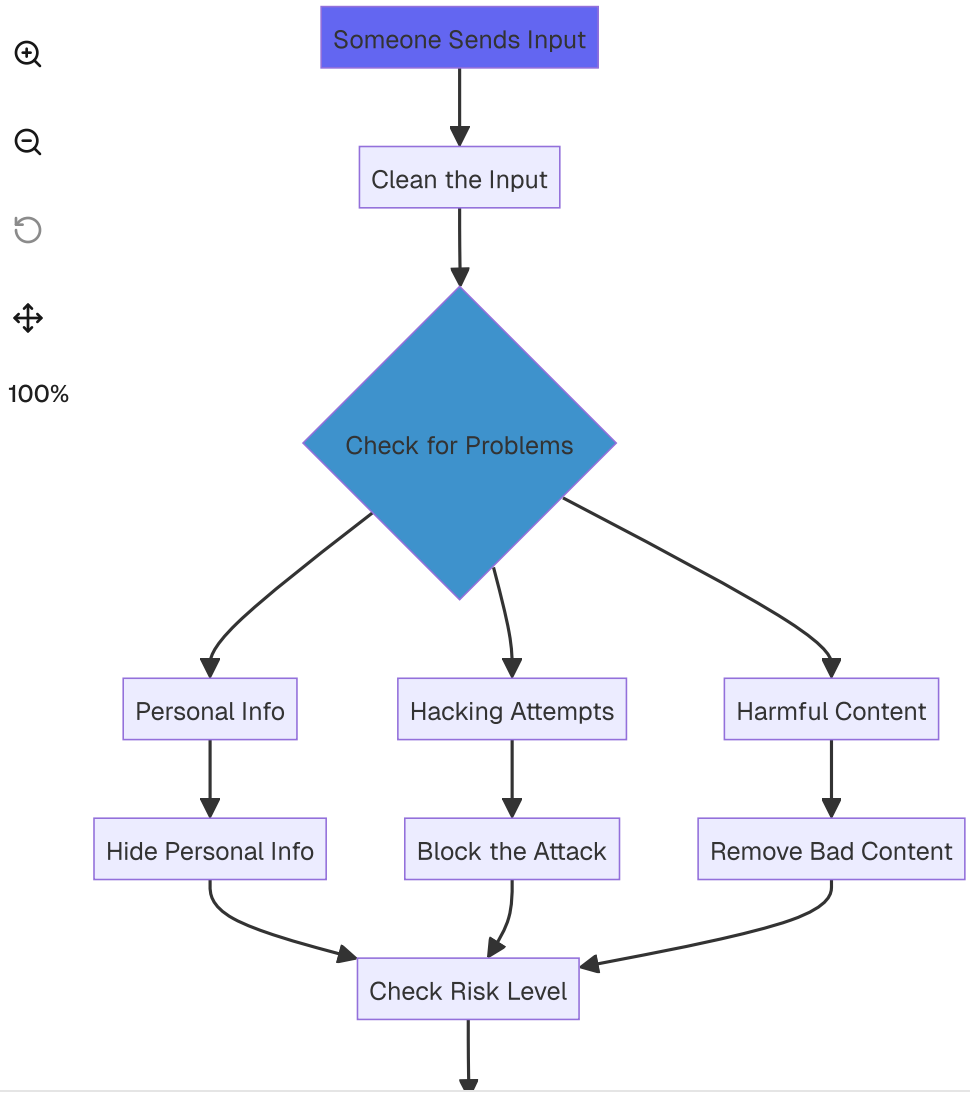
When to Use Where It Fits

When to Use

- **Public-facing systems**: Protecting users from harmful content
- **Regulated industries**: Ensuring compliance with laws
- **Brand protection**: Maintaining company reputation
- **Data privacy**: Protecting sensitive information
- **Security requirements**: Preventing system exploitation
- **Ethical AI**: Ensuring responsible AI behavior

How It Works

MermaidZoom MMD



How do guardrails and safety patterns work?

Clean input, check for problems - personal info, hacking attempts, harmful content. Hide personal info, block attacks, remove bad content

Then what?

What about output?

Check response against company rules - is it ethical? Legal? Match brand values? Calculate safety score

What if safety score is too low?

Check tool limits, use protected mode, check permissions. May need approval. Final decision: allow, fix the response, or block with explanation

How do you improve?

Record what happened, watch for patterns. Human can override and update rules, or system learns from patterns

Pros Cons

Pros

- **Risk mitigation**: Prevents harmful outputs
- **Compliance**: Meets regulatory requirements
- **Brand protection**: Maintains reputation
- **User safety**: Protects from inappropriate content
- **Security**: Prevents exploitation attempts
- **Consistency**: Uniform safety standards
- **Auditability**: Clear safety decision trails

Social Media Healthcare Financial Education Enterprise Content Generation

Social Media AI Moderator

- Hate speech detection and filtering
- PII redaction in user posts
- Misinformation flagging
- Violence/graphic content blocking
- Copyright violation detection
- Appeal process for false positives

Chapter 19: Prioritization

TLDDR: Scoring tasks based on value, risk, effort and urgency. The strategy in this pattern is you build something called a dependency graph to understand what needs to happen first. What in sequence needs to happen before the next following actions can follow after. Think of it like having an emergency room triage system that handles the most critical cases first, but it makes sure that everyone gets seen eventually.

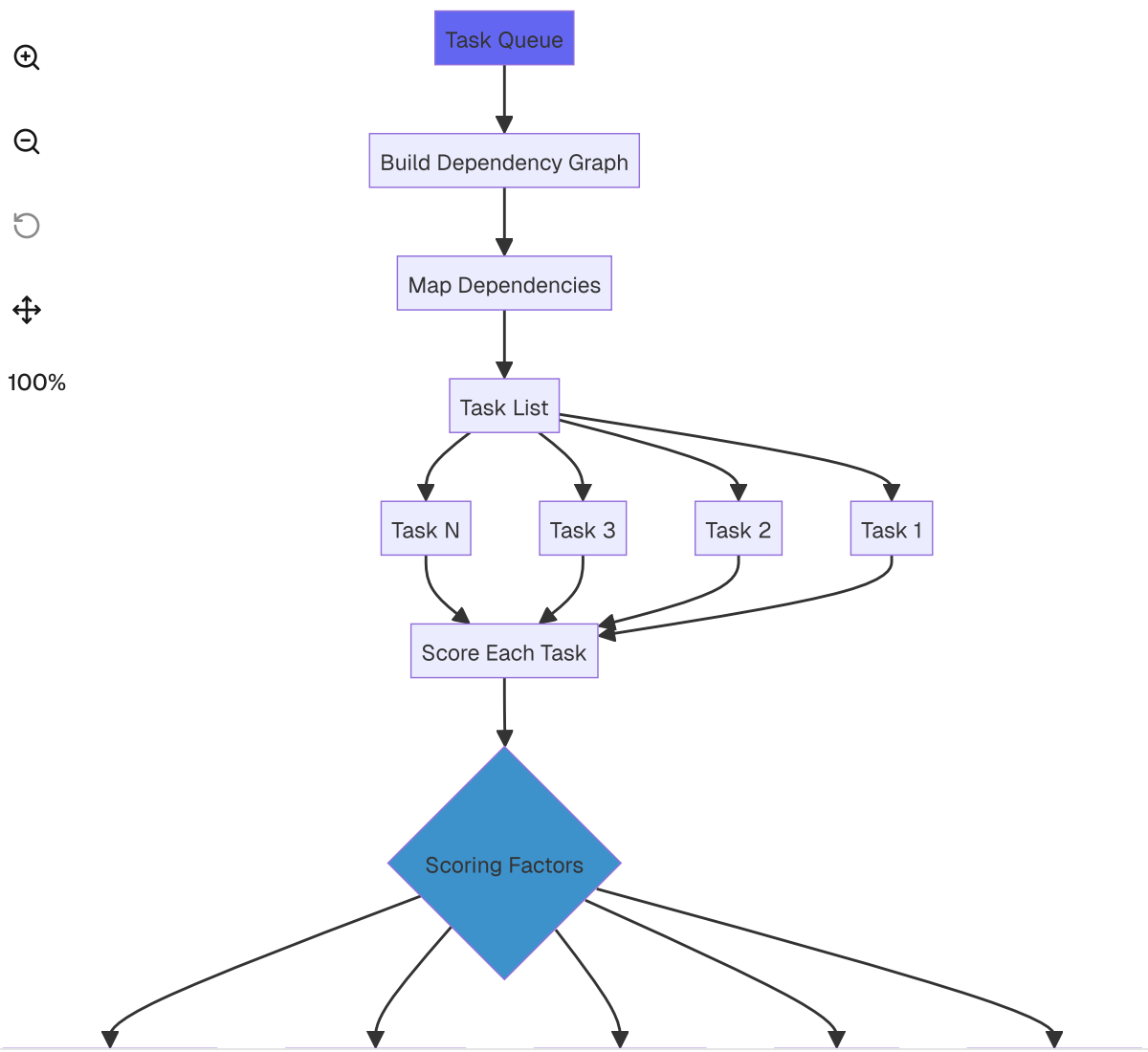
When to Use Where It Fits

When to Use

- **Resource constraints**: Limited processing capacity
- **Multiple objectives**: Competing goals and tasks
- **Dynamic environments**: Constantly changing priorities
- **Complex dependencies**: Tasks with interdependencies
- **Time-sensitive operations**: Deadline-driven work

How It Works

MermaidZoom MMD



How does prioritization work?

Build dependency graph - understand what needs to happen first. Map dependencies, create task list

Score based on business value, risk level, effort required, time sensitivity, dependency count. Calculate priority score = value/effort × urgency × risk

Then what?

Rank tasks, create initial order. Apply scheduling strategy - quotas (prevent starvation), task aging (boost old tasks), load balancing

What if priorities change?

Monitor for new high priority tasks. If yes, preempt current task, save state, switch to high priority. If no, continue current task

After task completes?

Remove from queue. Check if new events happened. If yes, re-calculate priorities. If no, get next task

Pros Cons

Pros

- **Efficiency**: Optimal use of resources
- **Responsiveness**: High-priority items handled first
- **Fairness**: Prevents indefinite delays
- **Adaptability**: Adjusts to changing conditions
- **Transparency**: Clear prioritization logic

Real-World Examples

Customer Support | Software | Healthcare | Manufacturing | Content Publishing | Network

Customer Support System

- Premium customers get priority
- Urgent issues ranked higher
- Age-based escalation
- Skill-based routing
- SLA compliance tracking
- Load balancing across agents

Chapter 20: Exploration and Discovery

TLDDR: Starting by broadly exploring the knowledge space across papers, data, and expert sources and identifying patterns and clustering them into themes. This one is like a detective gathering clues from everywhere, finding patterns, then focusing on the most promising leads. You can imagine this as the system responsible for things like perplexity deep research, claw deep research. Anything that has to go the next natural mile will take 40 minutes, spin up multiple agents to execute that research and scope out what is worth looking at versus what's not worth looking at.

When to Use | Where It Fits

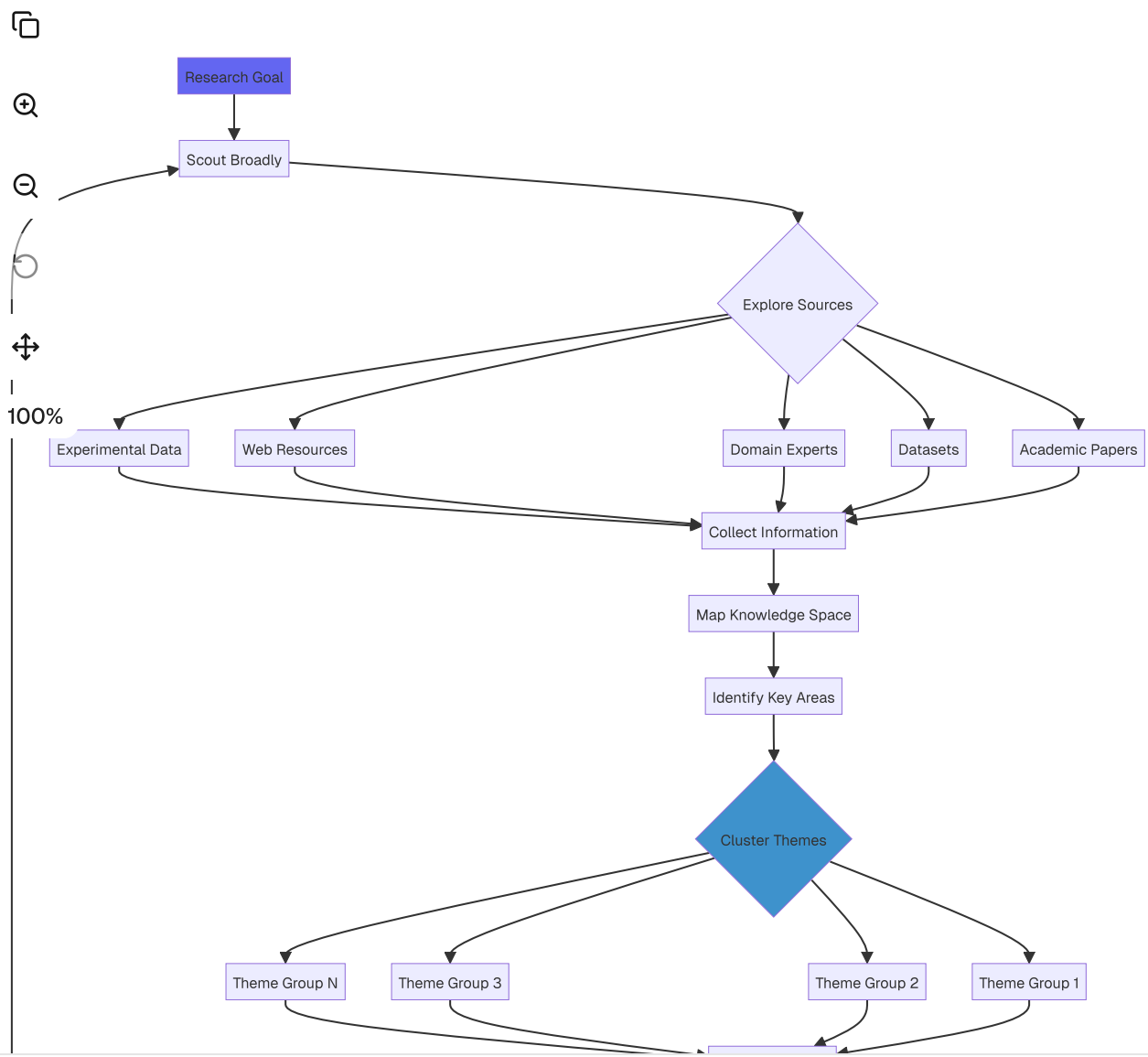
When to Use

- **Research projects**: Investigating new domains

- **Knowledge gaps**: Identifying what's unknown
- **Competitive analysis**: Discovering market opportunities
- **Scientific research**: Generating and testing hypotheses

How It Works

MermaidZoom MMD



Start with research goal, scout broadly. Explore sources - academic papers, datasets, domain experts, web resources, experimental data. Collect all information

Then what?

Map knowledge space, identify key areas. Cluster themes into groups. Analyze patterns across themes

How do you pick what to focus on?

Select deep-dive targets based on selection criteria - novelty score, potential impact, feasibility, knowledge gaps. Pick exploration targets

What happens in deep investigation?

Extract artifacts - research notes, bibliography, curated datasets, expert contacts, conceptual models. Synthesize insights

Built by neldivad. Copyright © 2024 - 2026 Nelworks Inc.

How do you finish?

Generate key insights, open questions, hypotheses. Check iteration limit - if not reached, design experiments, test hypotheses, gather results, loop back. If reached, conclude exploration, generate report, document findings, recommend next steps

Pros Cons

- **Innovation enablement**: Discovers new possibilities
- **Comprehensive coverage**: Broad exploration of space
- **Pattern recognition**: Identifies hidden connections
- **Hypothesis generation**: Creates testable theories
- **Knowledge building**: Accumulates domain expertise
- **Serendipity**: Enables unexpected discoveries
- **Systematic approach**: Structured exploration process

Real-World Examples

Drug Discovery Market SciResearch Tech Scout Intelligence EduResearch

Drug Discovery Platform

- Literature mining for drug targets
- Chemical space exploration
- Side effect pattern analysis
- Clinical trial data mining
- Hypothesis generation for compounds
- Experimental design optimization